# 2.8. Работа с датами и временем (datetime). Модули os  sys, безопасная работа с файловой системой. Описательная статистика

2. Файлы, данные, простая обработка и визуализация

# В этом уроке

- Работа с датами и временем (datetime)
- Модули os и sys
- Безопасная работа с файловой системой
- Описательная статистика в Python

# Datetime. Основы

Импорт модуля

```
import datetime
from datetime import date, time, datetime, timedelta
```

Классы модуля:

- date — представляет дату (год, месяц, день).
- time — представляет время (часы, минуты, секунды, микросекунды).
- datetime — объединяет дату и время в одном объекте.
- timedelta — разница между двумя датами/временем.

# Datetime. Работа с текущим временем

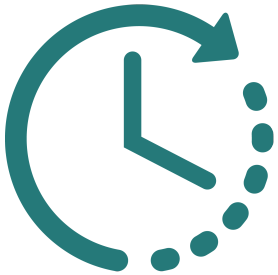
## Получение текущей даты и времени:

```
from datetime import datetime, date

now = datetime.now()   # текущие дата и время
today = date.today()   # только текущая дата
```

## Форматирование даты — strftime()

```
now.strftime("%Y-%m-%d %H:%M:%S")  # '2025-06-13 14:55:00'
now.strftime("%A, %d %B %Y")       # 'Friday, 13 June 2025'
```

| Код | Значение |
|---|---|
| %Y | Год (2025) |
| %m | Месяц (06) |
| %d | День (13) |
| %H:%M:%S | Время: часы, минуты, секунды |
| %A, %B | День недели и месяц текстом |



# Datetime. Операции с датами

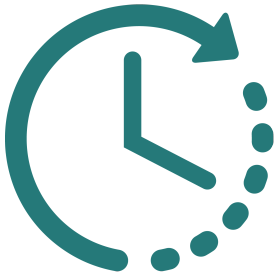
Разница между датами

```
from datetime import date


d1 = date(2025, 6, 13)
d2 = date(2020, 1, 1)
delta = d1 - d2          # результат: timedelta
print(delta.days)        # 1259
```

# Datetime. Операции с датами

Добавление и вычитание timedelta

```
from datetime import timedelta


tomorrow = date.today() + timedelta(days=1)
one_week_ago = date.today() - timedelta(weeks=1)
```



# Datetime. Операции с датами

## Вычисление возраста

```
birthdate = date(2000, 5, 1)
today = date.today()
age = today.year - birthdate.year - ((today.month, today.day) < (birthdate.month, birthdate.day))
```

# Datetime. Использование

Использование:

- Проверка сроков действия (акции, гарантии)
- Создание отчётов по временным периодам
- Автоматизация бэкапов и логов

# Что такое os?

Модуль os предоставляет функции для взаимодействия с операционной системой: навигация по файловой системе, управление файлами и папками.

- os.getcwd() — получить текущую рабочую директорию
- os.chdir(path) — сменить текущую директорию
- os.listdir(path) — список файлов и папок в директории
- os.remove(path) — удалить файл
- os.rename(src, dst) — переименовать/переместить файл
- os.makedirs(path) — создать директорию (включая промежуточны

# Что такое sys?

Модуль sys предоставляет доступ к переменным и функциям, используемым или управляемым интерпретатором Python

- sys.argv — аргументы командной строки
- sys.exit([code]) — завершение программы
- sys.path — список путей поиска модулей

# Основы безопасной работы с файлами

Зачем нужна безопасность при работе с файлами?

- Ошибки при открытии/удалении файлов могут вызвать сбои.
- Важно обрабатывать непредвиденные ситуации: отсутствующий файл, нет доступа, неправильный путь и т.д.

# Основы безопасной работы с файлами

Проверка существования файла или папки

```
import os

if os.path.exists("data.txt"):
    print("Файл найден")
else:
    print("Файл не существует")
```

Получение абсолютного пути

```
file_path = os.path.abspath("report.txt")
print(file_path)  # Например: /home/user/project/report.txt
```

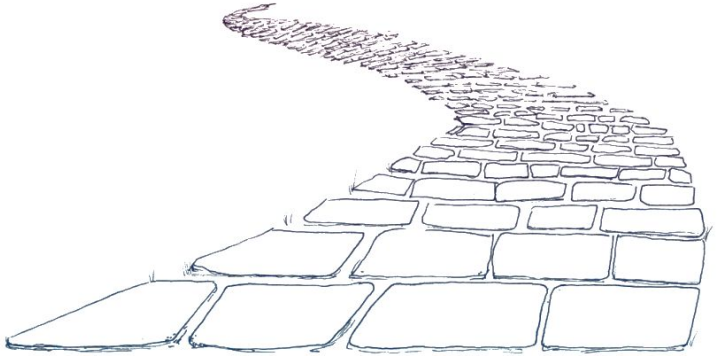
Обработка ошибок через try/except

```
try:
    with open("config.json", "r") as file:
        data = file.read()
except FileNotFoundError:
    print("Файл не найден.")
except PermissionError:
    print("Нет доступа к файлу.")
except Exception as e:
    print(f"Произошла ошибка: {e}")
```

# Работа с путями (модуль os.path)

Объединение путей

```
import os

folder = "backup"
filename = "2025-06-13.txt"
full_path = os.path.join(folder, filename)
print(full_path)  # backup/2025-06-13.txt
```



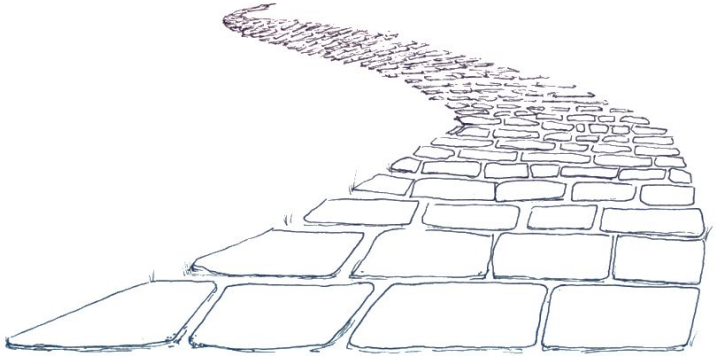
# Работа с путями (модуль os.path)

## Получение имени файла и расширения

```
path = "/home/user/archive/data.csv"

basename = os.path.basename(path)     # 'data.csv'
name, ext = os.path.splitext(basename)  # 'data', '.csv'
```

## Получение директории

```
directory = os.path.dirname(path)  # '/home/user/archive'
```



# Что такое описательная статистика

Задача описательной статистики - кратко описать свойства числовых данных: средние значения, распределение, разброс и отклонения

| Показатель | Что измеряет | Обозначение |
|---|---|---|
| Среднее арифметическое | Общее «среднее» значение | mean |
| Медиана | Центральное значение при сортировке | median |
| Мода | Наиболее часто встречающееся значение | mode |
| Дисперсия | Насколько значения разбросаны от среднего | variance |
| Стандартное отклонение | Среднее отклонение от среднего | stdev |

# Модуль statistics

```
import statistics as stats
```

| Функция | Назначение |
|---|---|
| mean(data) | Среднее арифметическое |
| median(data) | Медиана |
| mode(data) | Мода |
| variance(data) | Дисперсия |
| stdev(data) | Стандартное отклонение |

# Модуль statistics

```
import statistics as stats
```

| Функция | Назначение |
|---|---|
| mean(data) | Среднее арифметическое |
| median(data) | Медиана |
| mode(data) | Мода |
| variance(data) | Дисперсия |
| stdev(data) | Стандартное отклонение |

# Модуль statistics

📈 **Применение в жизни:**

- 📊 Оценка успеваемости студентов
- ⚙️ Контроль качества продукции
- 💬 Анализ пользовательского поведения
- Обработка результатов экспериментов

⚠️ **Возможные подводные камни:**

- Наличие выбросов может сильно исказить среднее
- Иногда медиана лучше отражает «типичное» значение
- Маленькие выборки дают ненадёжные оценки дисперсии и отклонения

# Итоги урока

- Разобрались с датами и временем (datetime)
- Изучили модули os и sys
- Узнали о безопасной работе с файловой системой
- Описательная статистика в Python